Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

MAKESH KUMAR D
MANIKANDAN R
MONILSON K
KUMARAN B
SAKTHI SREE B

INTRODUCTION

In the contemporary digital ecosystem, organizations are continuously confronted with sophisticated cyber threats and security breaches. As technology evolves rapidly, attackers exploit vulnerabilities to gain unauthorized access to sensitive systems and confidential data. Consequently, organizations must adopt advanced security mechanisms to detect, prevent, and respond to cyber incidents efficiently. The necessity for intelligent and automated security frameworks has therefore become a fundamental requirement for protecting digital infrastructures and maintaining the trust of stakeholders.

Traditionally, organizational security relied on the perimeter-based security model, commonly referred to as the “castle-and-moat” approach. In this model, users inside the corporate network were inherently trusted, while external users were restricted. However, the increasing adoption of cloud computing, remote working environments, and Bring Your Own Device (BYOD) policies has significantly diminished the effectiveness of this model. Corporate resources are now accessed from diverse locations, devices, and networks, which makes static security models inadequate for modern enterprise environments.

To overcome these limitations, organizations are shifting toward identity-centric and contextaware security architectures. These modern frameworks evaluate multiple contextual parameters before granting access to organizational resources. Instead of relying solely on username and password authentication, advanced access control systems analyze factors such as user identity, device security posture, geographical location, and application sensitivity level.

This project focuses on the design and implementation of a context-aware access control mechanism that enables intelligent and adaptive decision-making for access requests. The system evaluates contextual signals including the user's identity and organizational role, the compliance level of the accessing device, the geographical origin of the request, and the risk classification of the requested application. Based on these parameters, the system dynamically determines the appropriate level of access.

For example, when a trusted employee accesses a low-risk application from a secured corporate device within the office network, the system allows seamless access without additional verification. Conversely, when the same user attempts to access a high-risk application from an unsecured device or an unfamiliar location, the system may enforce additional security measures such as multi-factor authentication or restrict the access entirely.

ABSTRACT

The rapid expansion of cloud technologies, mobile computing platforms, and hybrid work environments has fundamentally transformed the modern digital infrastructure. As a result, the traditional network perimeter has become increasingly ineffective in safeguarding organizational resources. Conventional access control models that rely on static network boundaries are no longer capable of addressing the complex security challenges faced by modern enterprises.

In contemporary distributed environments, cyber threats may originate from compromised devices, untrusted network locations, or even malicious insiders. Therefore, simple binary access decisions such as “allow” or “deny” are insufficient for ensuring robust security. Organizations now require dynamic and intelligent security frameworks that can evaluate multiple contextual parameters before granting access to sensitive systems and applications.

This project proposes the development and implementation of a context-aware access control system that utilizes multiple contextual signals including user identity, device posture, geographical location, and application risk level. The primary objective of the system is to analyze these contextual attributes and generate a real-time risk assessment for every access request.

The system incorporates a policy-driven decision engine that maps different contextual scenarios to appropriate access decisions. For instance, when a trusted user accesses a low-risk application from a compliant corporate device, the system grants seamless access. However, if the same user attempts to access a sensitive application from an untrusted location or noncompliant device, the system may enforce additional security measures such as multi-factor authentication, restricted session capabilities, or complete access denial.

The expected outcome of this project is the development of a flexible and adaptive security architecture that significantly reduces the organizational attack surface while maintaining operational efficiency. By continuously evaluating contextual risk factors, the system ensures that access decisions remain aligned with the real-time security posture of the environment.

This approach not only strengthens the protection of critical resources but also enables organizations to maintain a secure yet frictionless user experience for employees, partners, and remote workers in a highly dynamic digital landscape.

REPORT (Sample – Improved Version)

Vulnerability Name: Stored Cross-Site Scripting (XSS)

CWEID:79
OWASP Category: A03:2021 – Injection

Description

Stored Cross-Site Scripting (XSS) is a web security vulnerability that occurs when a web application stores malicious scripts in a persistent storage location such as a database, message forum, or comment section. These scripts are later executed in the browser of other users when the stored data is retrieved and displayed by the application.

Unlike reflected XSS, where the malicious script is immediately returned in the server response, stored XSS remains permanently embedded within the application's data repository. When unsuspecting users access the compromised page, the malicious script is executed automatically within their browser environment.

Business Impact

The presence of stored XSS vulnerabilities can have severe consequences for organizations. Attackers can exploit this vulnerability to perform unauthorized actions on behalf of legitimate users, steal session cookies, capture sensitive information, or redirect users to malicious websites.

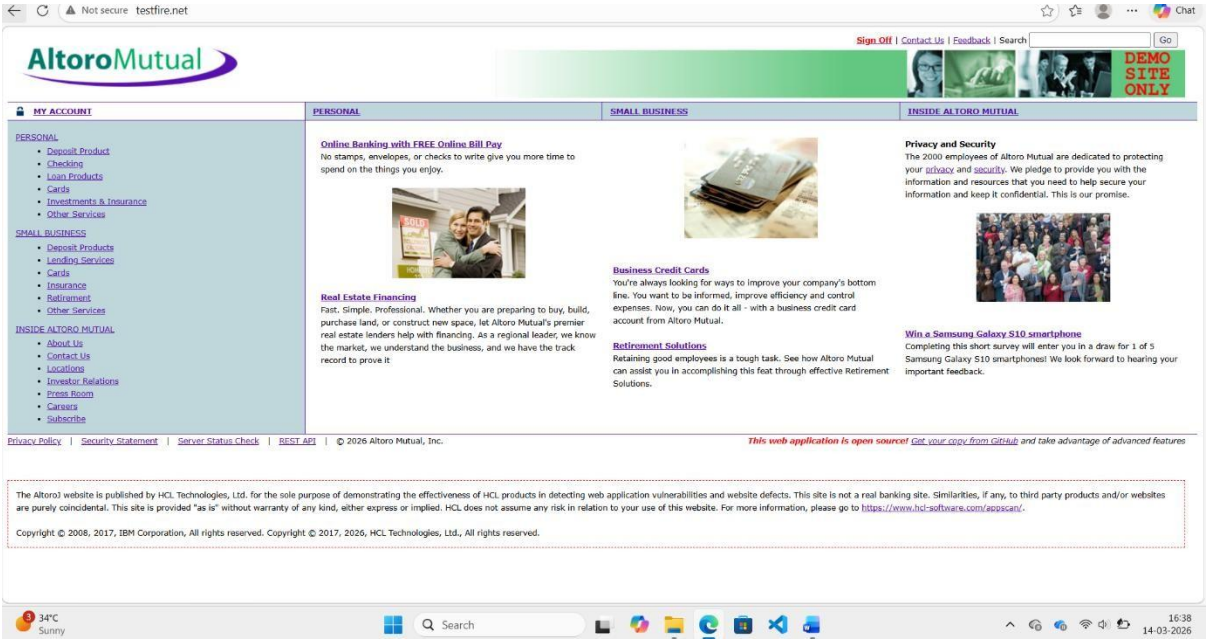
Additionally, if privileged users such as administrators execute the malicious script, attackers may gain elevated access to critical system functionalities and confidential organizational data. This can ultimately result in data breaches, financial losses, reputational damage, and legal implications for the organization.

Vulnerability path: http://testfire.net/

Vulnerability parameter:
http://testfire.net/search.jsp?query=%27%3E%3Cscript%3Ealert%28%27hacked%27%29%3C%2Fscrip t%3E

Steps to Reproduce:

➢ Access the URL



➢ In the search box we will input some code to perform the vulnerability. ➢ The Script we will be inputting is '>

➢ This displays a harmless pop up alert box with the text saying 'hacked'

Recommendation:

- When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.
- Sanitize and validate all user inputs and ensure that any data displayed on the web page is properly encoded. Use output encoding libraries or functions to prevent script injection.

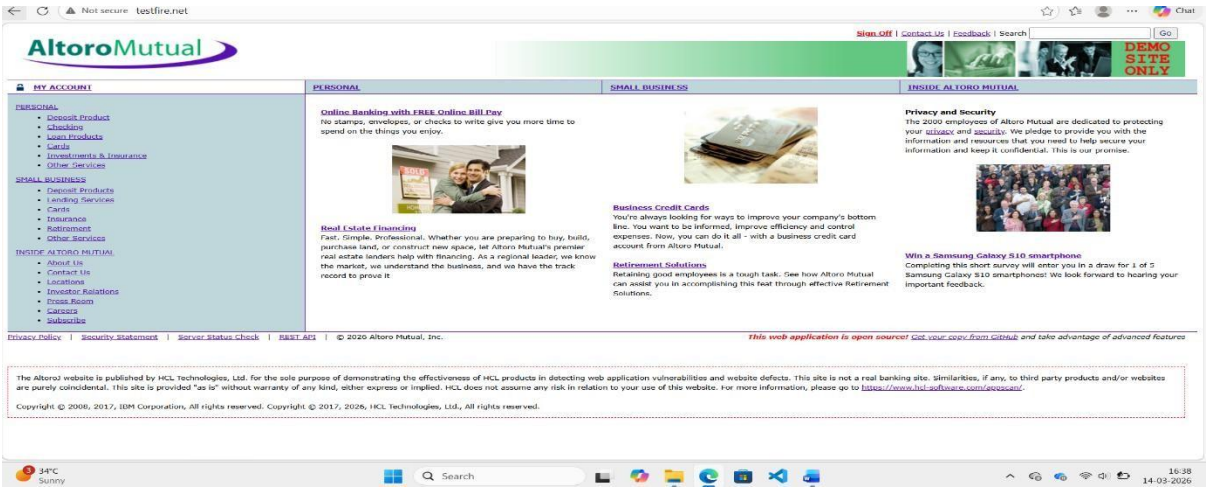
2. Vulnerability name: Broken access control CWE: 285 Description: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution. Business Impact: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects

Vulnerability path: http://testfire.net/

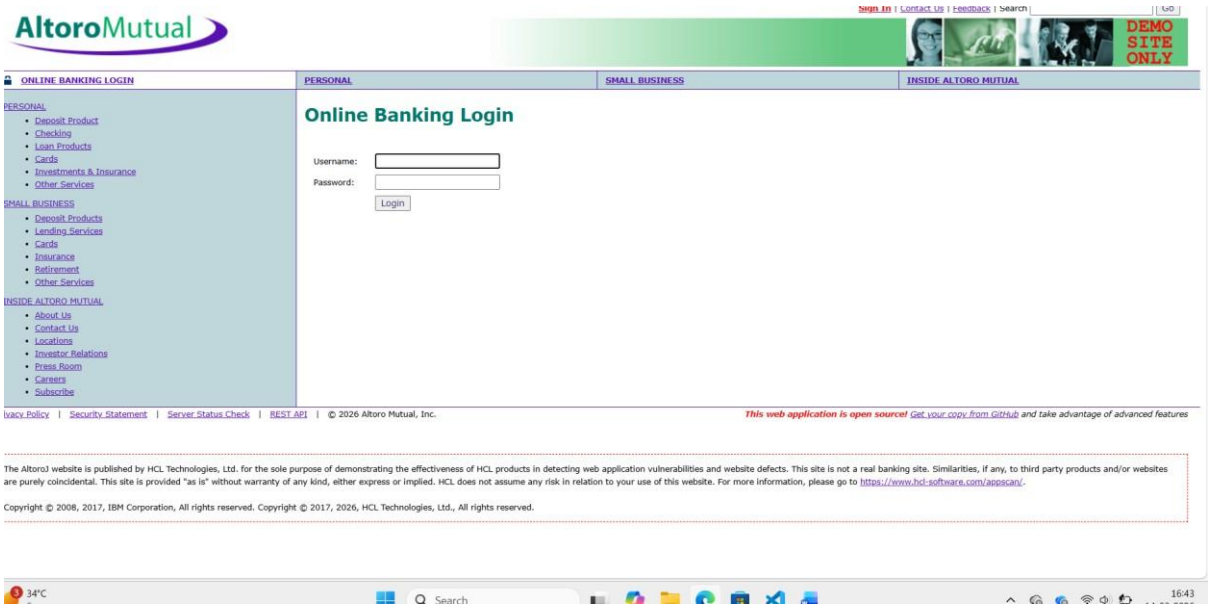
Vulnerability parameter : http://testfire.net/login.jsp

Steps to Reproduce:

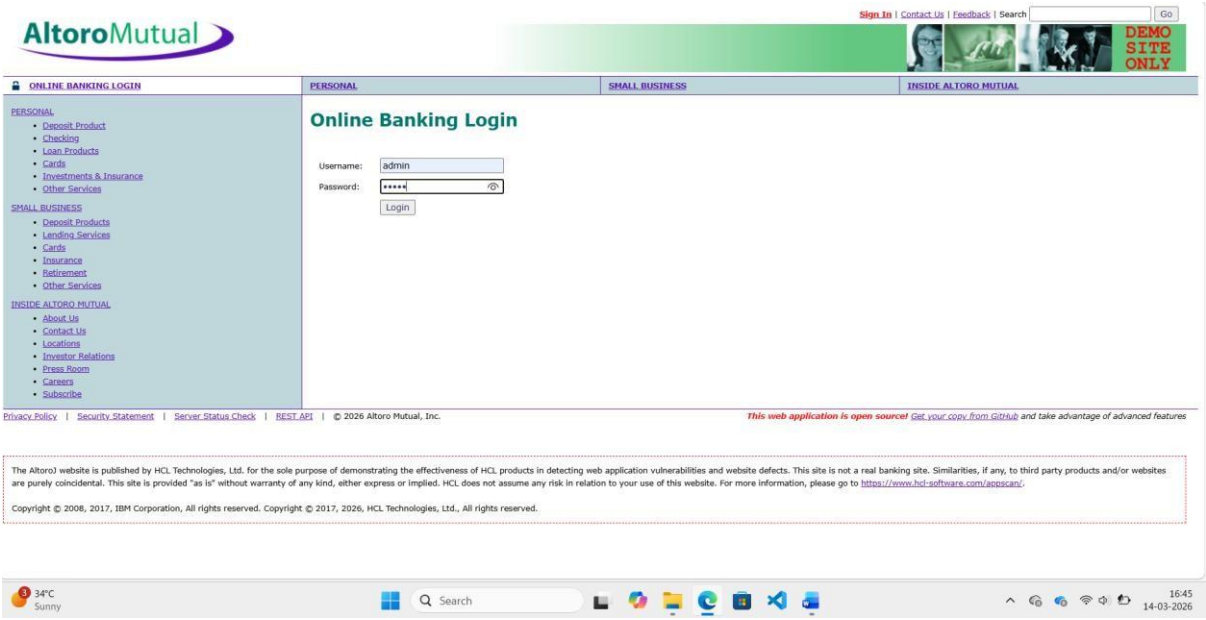
➢ Access the URL



➢ Now we will try to sign in to this website with admin privileges

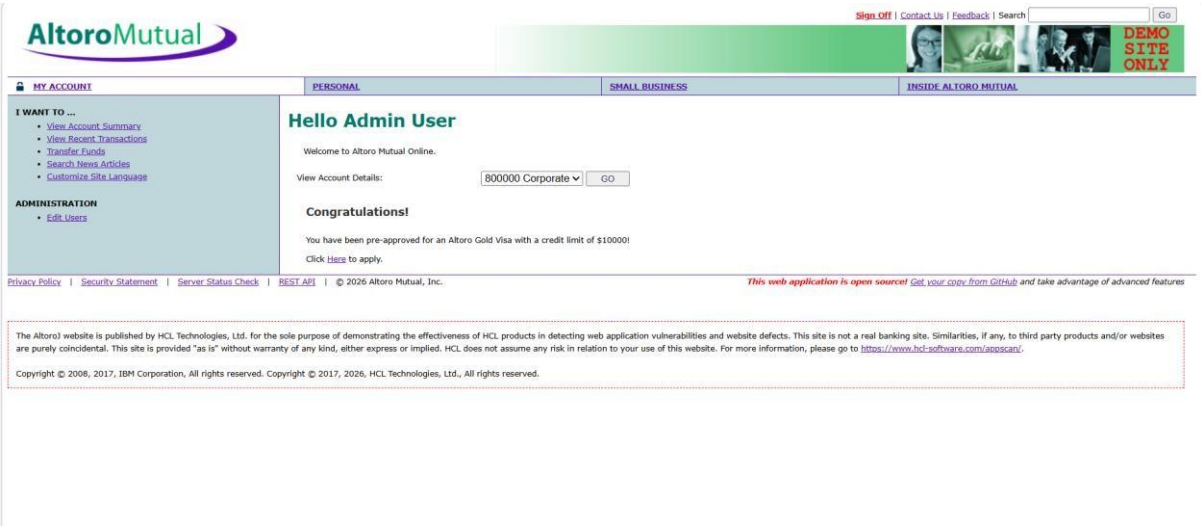


➢ Also, we will be using burp suite to get requests from the website and know additional information. We use ‘admin’ for the username and password.

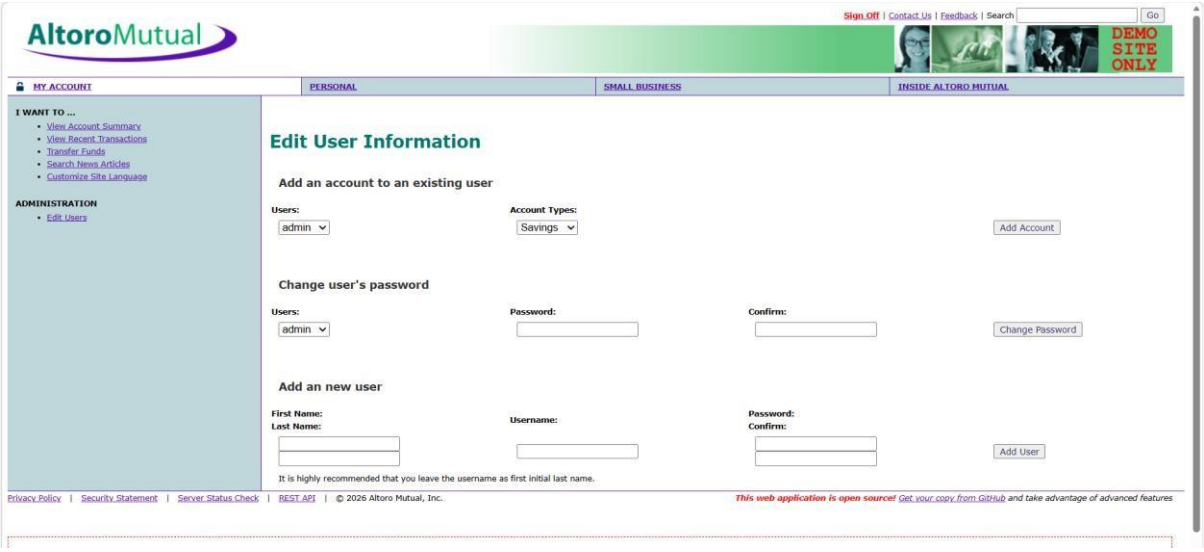


➢ This request has been received in the burp suite with the username and password as well in clear text.

➢ now we click on forward request in the burp suite and then we will be redirected to the admin user details.



➢ We can even edit the other users data and delete their login details as well.



Recommendations:

- Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.

- Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

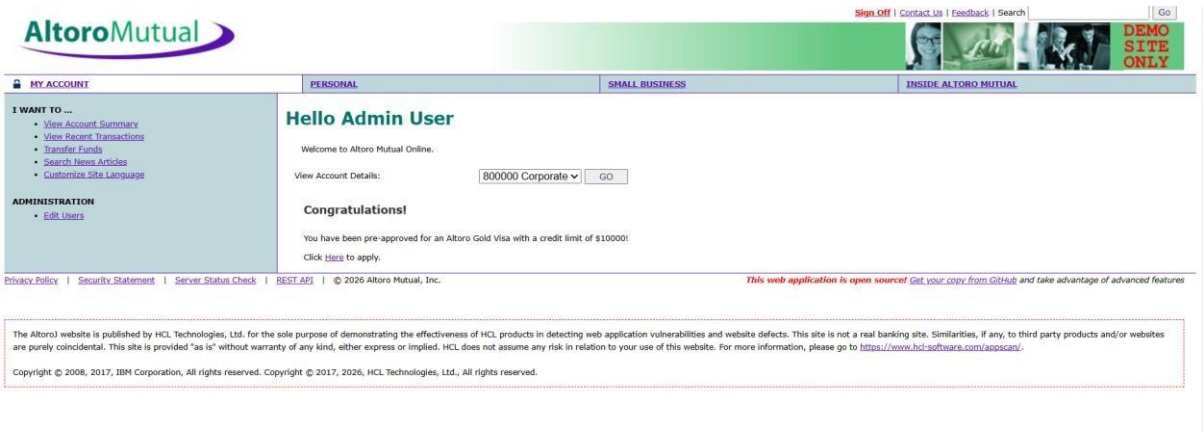
3. Vulnerability name: SQL injection CWE: 89 Description: The product constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component. Business Impact: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil. Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

Vulnerability path: http://testfire.net/

Vulnerability parameter: http://testfire.net/login.jsp

Steps to Reproduce:

- ➢ Access the URL
- ➢ Now we will try to sign in to this website with admin privileges but using SQL injection
- ➢ Also, we will be using burp suite to get requests from the website and know additional information.
- ➢ Click forward request multiple times to proceed to the login page.
- ➢ With this we can know that sql injection worked and we got the admin privileges.



Recommendations:

- If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.
- Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.
- Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

4. Vulnerability name: Broken authentication CWE: 285 Description: The system's authorization functionality does not prevent one user from gaining access to another user's data

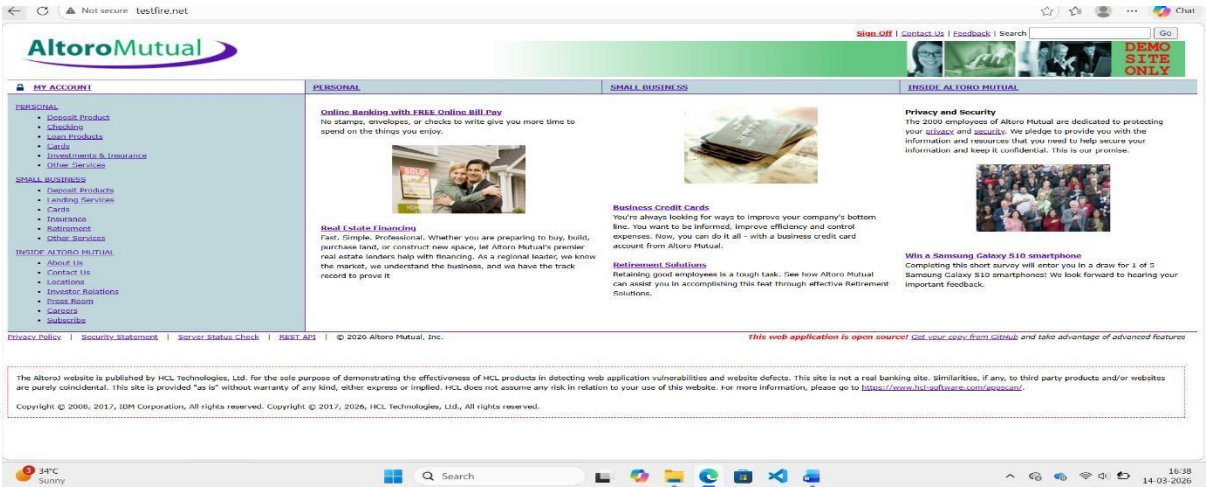
or record by modifying the key value identifying the data. Business Impact: To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security assessments and testing. The rectification of these vulnerabilities stands as a critical imperative, safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

Vulnerability path: http://testfire.net/

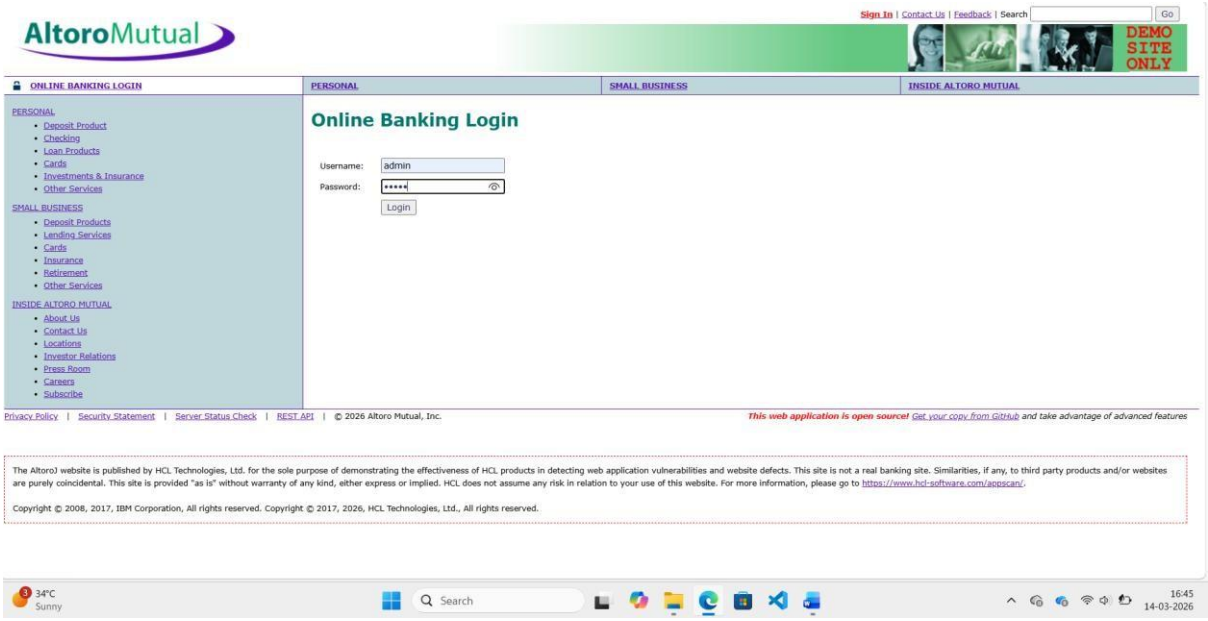
Vulnerability parameter: http://testfire.net/login.jsp

Steps to Reproduce:

- Access the URL



- Now we will try to login using some different approach.



- As we know which users are present in the database of this website by using the admin privileges. We can directly access a particular user by simply knowing their username; we will add some characters after his user name as a sql injection to simply bypass the password.

- This leads us to the details of this person's account.

Recommendations:

- Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or two- factor authentication (2FA), to ensure that only authorized users can access the system.

- Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.

- Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

5. Vulnerability Name: Insecure Direct object Reference CWE: 639 OWASP Category: A01: Broken Access Control Description: Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's URLs or parameters. It occurs due to missing

access control checks, which fail to verify whether a user should be allowed to access specific data. Business Impact: To effectively mitigate the business impact of CWE639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as absolutely critical in the defense of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

Vulnerability Path: http://testfire.net

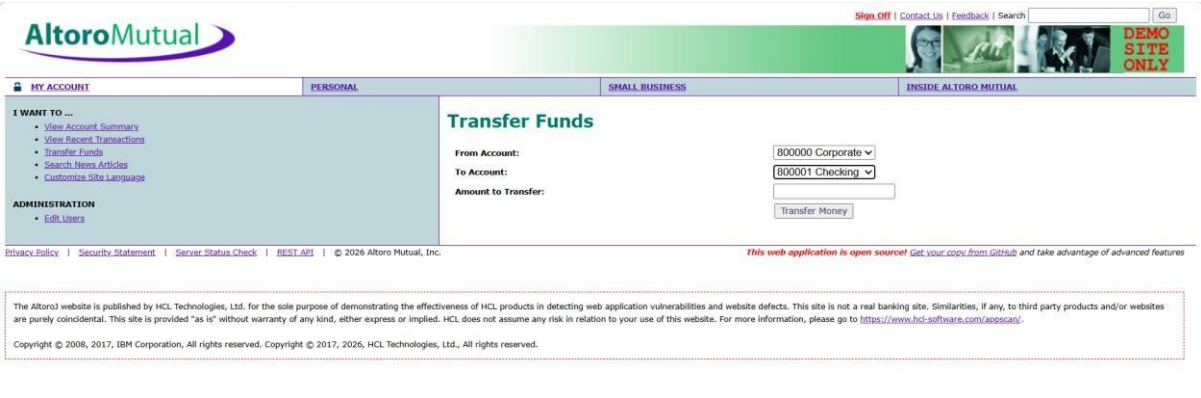
Vulnerability Parameter : http://testfire.net/bank/transfer.jsp **Steps**

to Reproduce:

➢ Go to the domain.

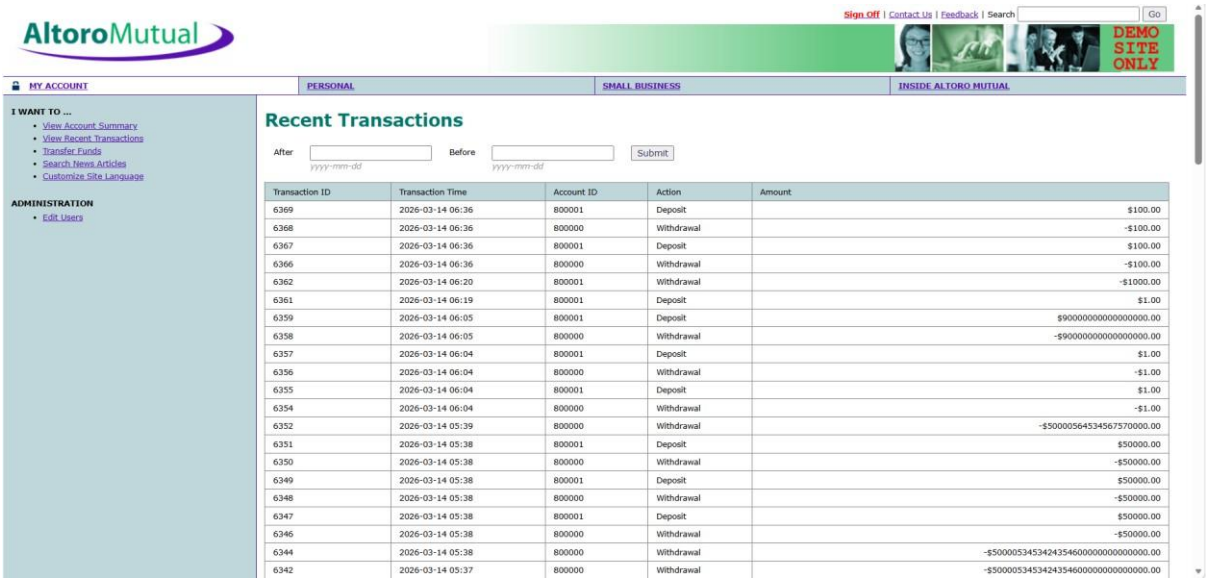
➢ Sign into the site.

➢ Open “Transfer Money” on the left side and fill in the details. On the intercept and click transfer



➢ Look at the site, we can notice the msg that shows the transfer of 1000.

➢ Off the Intercept and open “View Recent Transactions”.



Transaction ID	Transaction Time	Account ID	Action	Amount
6369	2026-03-14 06:36	800001	Deposit	$100.00
6368	2026-03-14 06:36	800000	Withdrawal	-$100.00
6367	2026-03-14 06:36	800001	Deposit	$100.00
6366	2026-03-14 06:36	800000	Withdrawal	-$100.00
6362	2026-03-14 06:20	800001	Withdrawal	-$1000.00
6361	2026-03-14 06:19	800001	Deposit	$1.00
6359	2026-03-14 06:05	800001	Deposit	$90000000000000000.00
6358	2026-03-14 06:05	800000	Withdrawal	-$90000000000000000.00
6357	2026-03-14 06:04	800001	Deposit	$1.00
6356	2026-03-14 06:04	800000	Withdrawal	-$1.00
6355	2026-03-14 06:04	800001	Deposit	$1.00
6354	2026-03-14 06:04	800000	Withdrawal	-$1.00
6352	2026-03-14 05:39	800000	Withdrawal	-$500005645345675700000.00
6351	2026-03-14 05:38	800001	Deposit	$50000.00
6350	2026-03-14 05:38	800000	Withdrawal	-$50000.00
6349	2026-03-14 05:38	800001	Deposit	$50000.00
6348	2026-03-14 05:38	800000	Withdrawal	-$50000.00
6347	2026-03-14 05:38	800001	Deposit	$50000.00
6346	2026-03-14 05:38	800000	Withdrawal	-$50000.00
6344	2026-03-14 05:38	800000	Withdrawal	-$500005345342435460000000000000.00
6342	2026-03-14 05:37	800000	Withdrawal	-$500005345342435460000000000000.00

Recommendation:

- Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.
- Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.

- Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

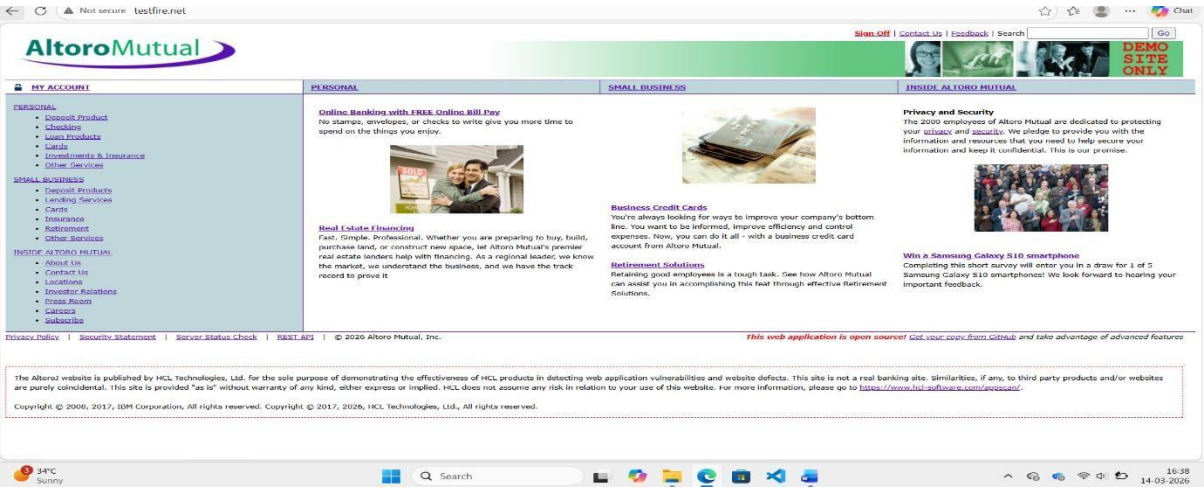
6. Vulnerability name: Security Misconfiguration CWE:732 Description: The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors. Business Impact: To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

Vulnerability path: http://testfire.net/

Vulnerability parameter: http://testfire.net/login.jsp Steps

to Reproduce:

➢ Access the URL



➢ We will be using the uniscan tool which is available in the kali.

```
┌──(kali㉿kali)-[~]
└─$ sudo uniscan -h
[sudo] password for kali:
#####################################
# Uniscan project                   #
# http://uniscan.sourceforge.net/   #
#####################################
V. 6.3

OPTIONS:
        -h      help
        -u      <url> example: https://www.example.com/
        -f      <file> list of url's
        -b      Uniscan go to background
        -q      Enable Directory checks
        -w      Enable File checks
        -e      Enable robots.txt and sitemap.xml check
        -d      Enable Dynamic checks
        -s      Enable Static checks
        -r      Enable Stress checks
        -i      <dork> Bing search
        -o      <dork> Google search
        -g      Web fingerprint
        -j      Server fingerprint

usage:
[1] perl ./uniscan.pl -u http://www.example.com/ -qweds
[2] perl ./uniscan.pl -f sites.txt -bqweds
[3] perl ./uniscan.pl -i uniscan
[4] perl ./uniscan.pl -i "ip:xxx.xxx.xxx.xxx"
[5] perl ./uniscan.pl -o "inurl:test"
[6] perl ./uniscan.pl -u https://www.example.com/ -r
```

➢ Here we can see the sensitive data of job applications which can help an attacker.

Recommendations:

- Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.
- Follow security guidelines provided by software and hardware vendors to ensure secure configurations.
- Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

7. Vulnerability Name: Cross-Site Request Forgery CWS: 352 OWASP

Category: A08: Cross-Site Request Forgery Description: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF

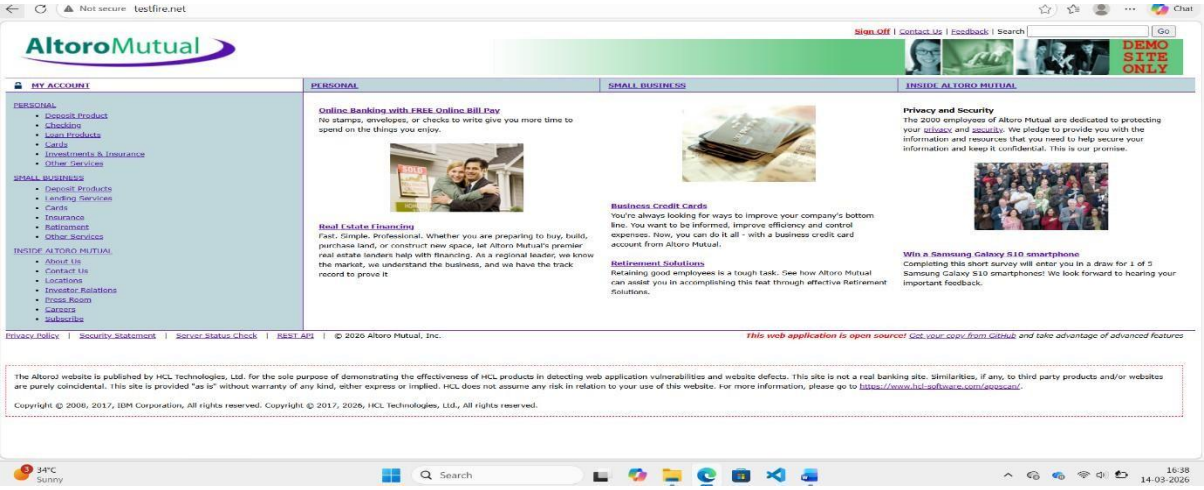
attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application. Business Impact: To effectively reduce the business repercussions of CWE-352, organizations must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities. Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

Vulnerability Path: http://testfire.net/

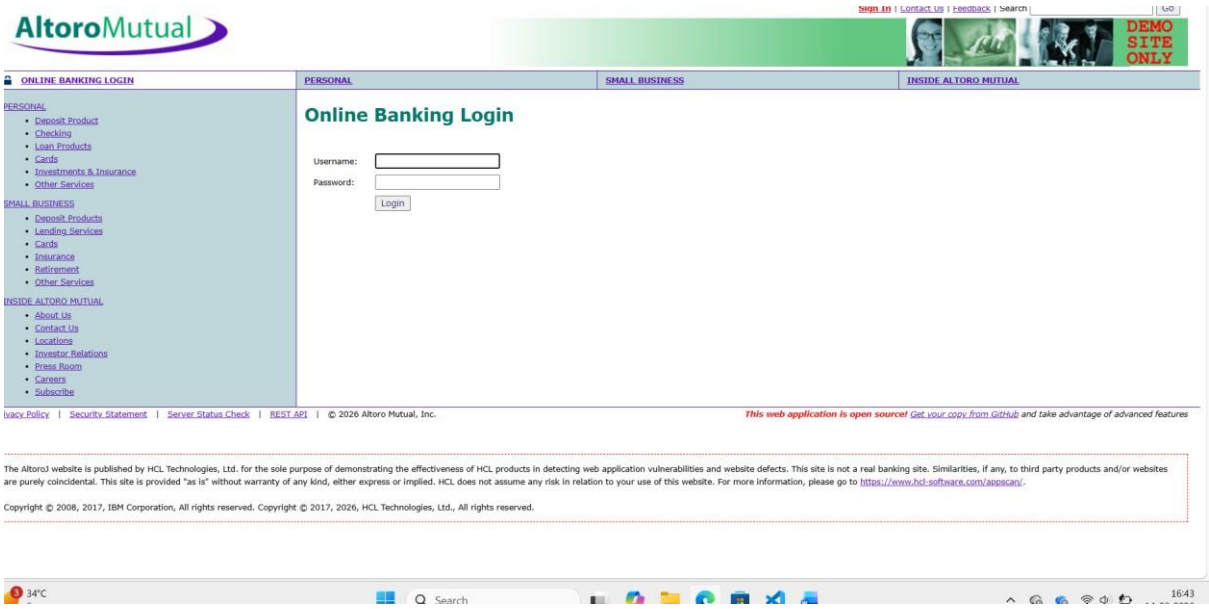
Vulnerability Parameter: http://testfire.net/bank/transfer.jsp Steps

to Reproduce:

- Open https://testfire.net/.

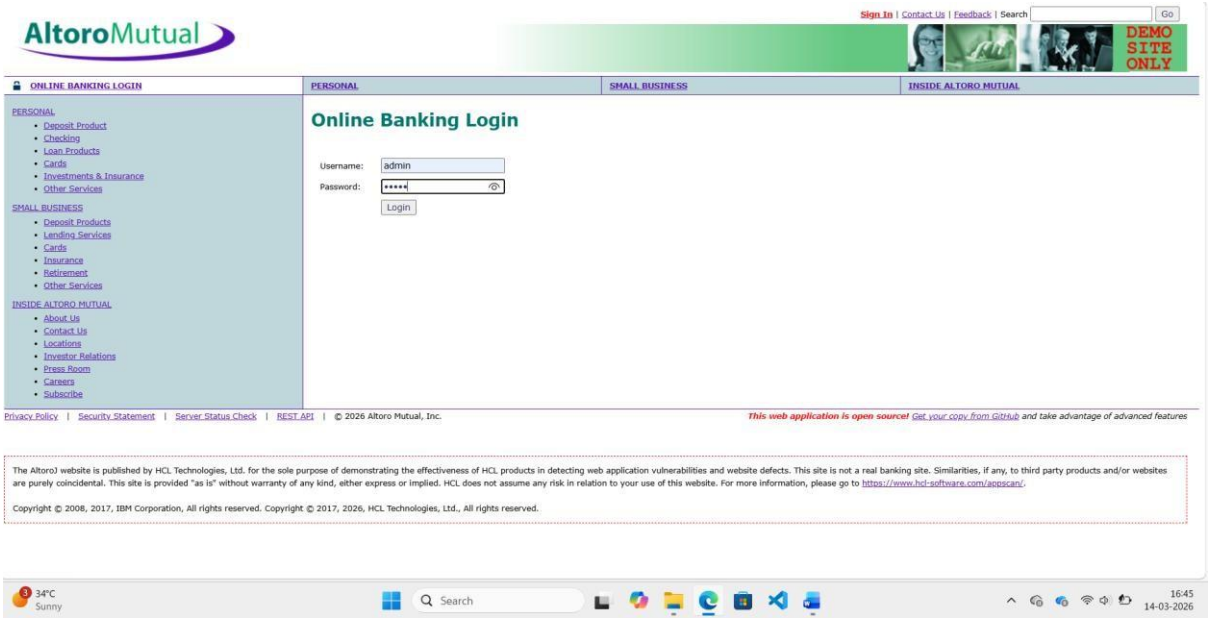


➢ Login to it using default credentials, and you will find two accounts.



➢ Capture the request in any Intercepting proxy (Burp Suite, Charles Proxy etc.) while transferring the amount from one account to another as shown in the below picture.

➢ Generate CSRF POC using Burp Suite Engagement tools (by modifying the account numbers accordingly)



➢ As the victim is already logged-in to the site an attacker can send the crafted malicious form/URL to the user to his email/text/by any other means and trick the victim using his social engineering skills/techniques like below.

➢ User thinks, he could use a free $100 gift card! What could go wrong? You open the email and click the link/open the attachment.

➢ Once the user opens the form a transaction is automatically performed from his account without his knowledge as the user is already logged in.

Recommendations:

- Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.
- Utilize the SameSite attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.
- Utilize framework-specific protections and security features to prevent CSRF attacks. Many modern web frameworks have built-in mechanisms and libraries for handling CSRF vulnerabilities effectively.
- Implement a Content Security Policy to restrict the sources from which various types of content can be loaded. By specifying the trusted sources of content, you can minimize the risk of malicious code execution and reduce the likelihood of successful CSRF attacks.

8. Vulnerability name: Cleartext Transmission of Sensitive Information CWE: 319

Description: The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

Business Impact: To effectively reduce the business consequences associated with CWE-319, organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols.

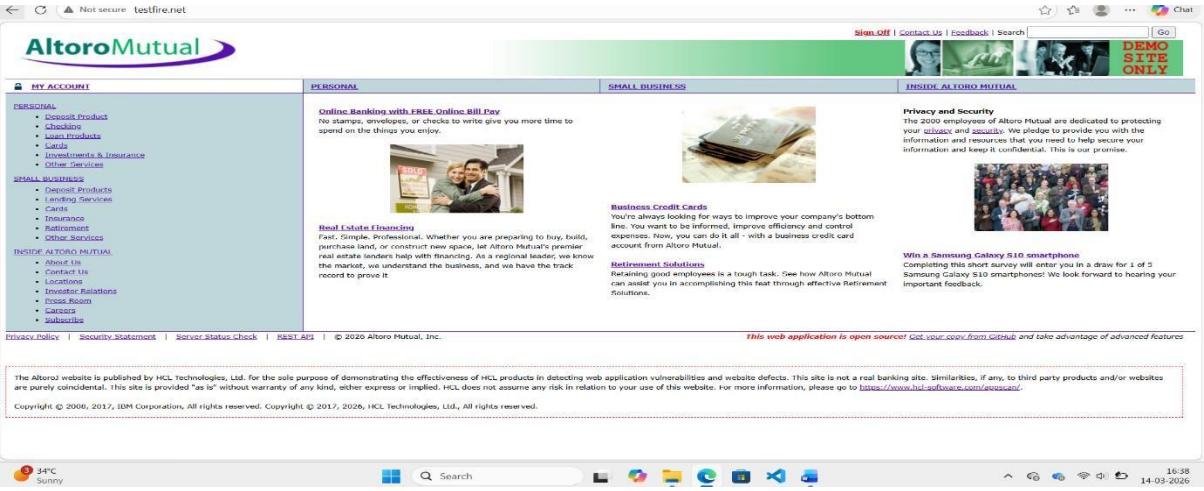
The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

Vulnerability path: http://testfire.net/

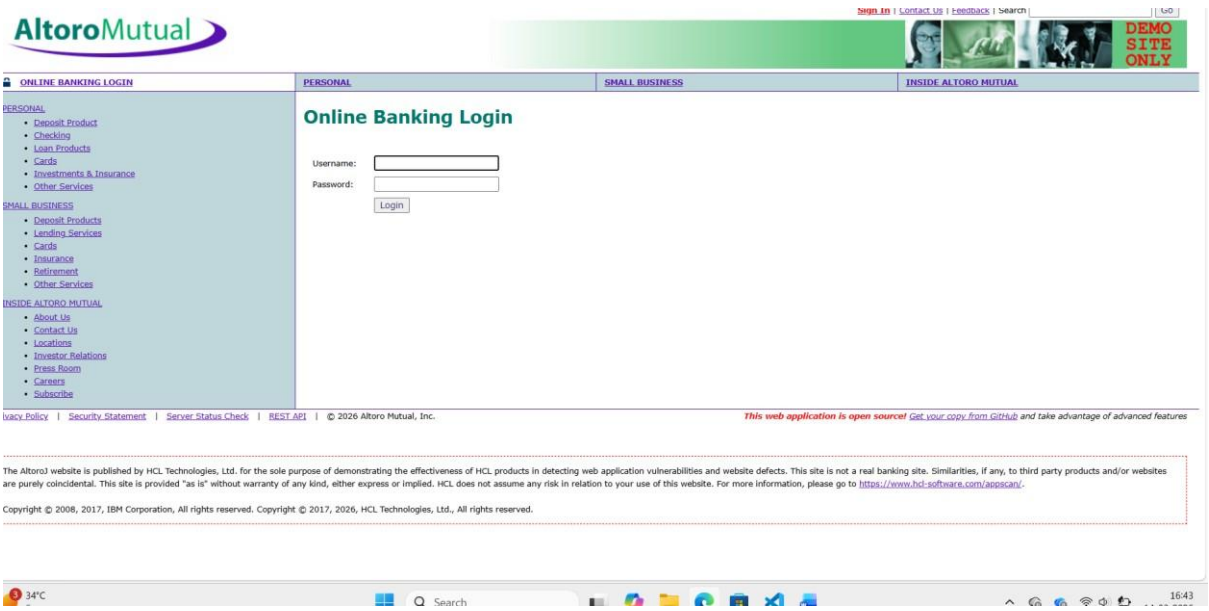
Vulnerability parameter: http://testfire.net/login.jsp Steps

to reproduce:

- ➢ Access the URL

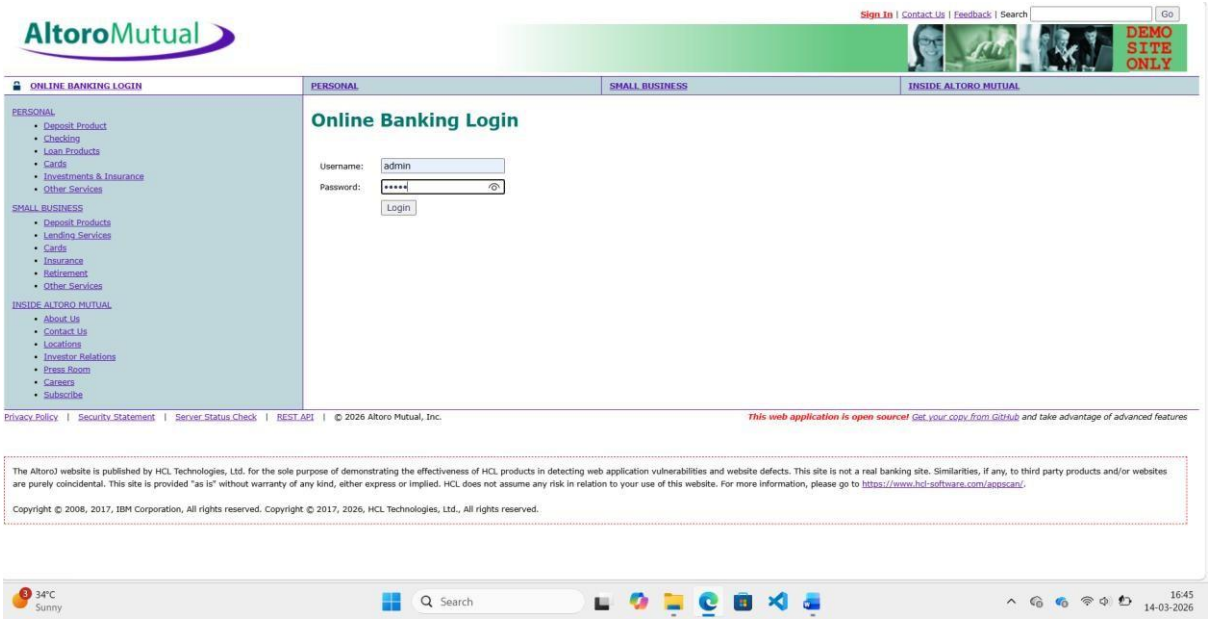


- ➢ Now we will try to sign in to this website with admin privileges



➢ Also, we will be using burp suite to get requests from the website and know additional information.

➢ We use ‘admin’ for the username and password.



➢ This request has been received in the burp suite with the username and password as
well in clear text.

➢ Now we click on forward request in the burp suite and then we will be redirected to
the admin user details. Here in the burp suite, we can clearly see the login details in clear text.

this is the clear indication of the vulnerability which can lead to data breach, monitored, and manipulated as well.

Recommendations:

- Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.
- Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission.
- Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.
- Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up-to-date.

9. Vulnerability name: Click Jacking CWE: 1021

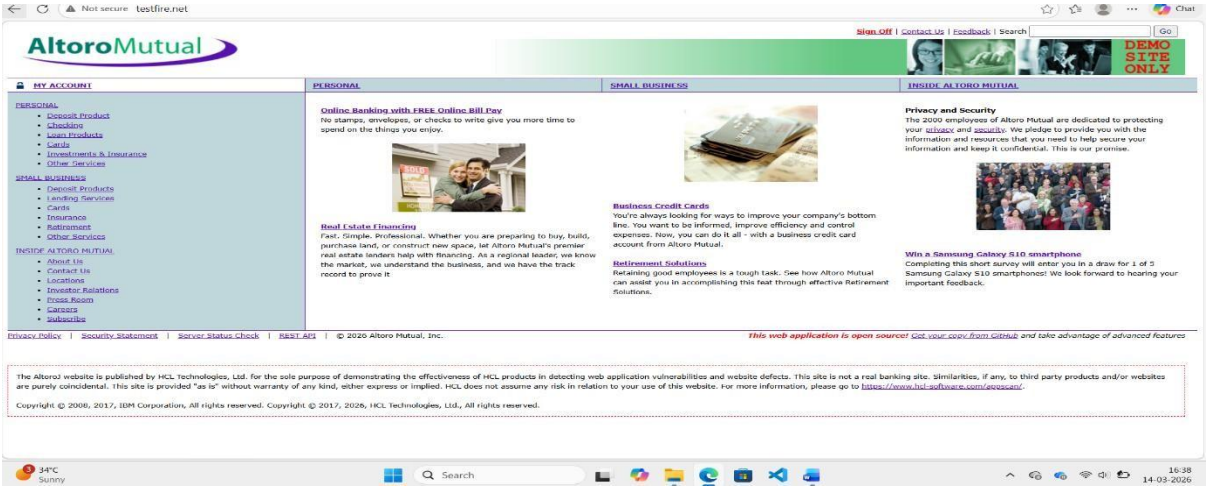
Description: The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

Business Impact: To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

Vulnerability path: http://testfire.net/ Vulnerability parameter: http://testfire.net/

Steps to reproduce:

- ➢ Access the URL



➢ Then take the URL and use it for the html code

➢ we will be writing some html code to perform this vulnerability. we will be writing the code in vs code for better flexibility and functionality.

```
<html>
<body>
<title>Click jacking vulnerability</title>
<h2>This website is vulnerable to clickjacking</h2>
<iframe  src="http://testfire.net/"></iframe>
</body>
</html>
```

➢ After the code has been written then we will be executing the code in the browser.

➢ From this image we can see that the vulnerability has been found.

Recommendation:

- Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers.
- Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious iframes. This code can disrupt clickjacking attempts.
- Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.
- Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

10. Vulnerability name: web server allows password auto-completion CWE: 310

Description: Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

Business impact: To effectively lessen the business consequences associated with CWE-310, organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

11. Vulnerability name: Cross-site scripting (DOM based XSS) CWE: 79 OWASP category: A03: Injections Description: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.

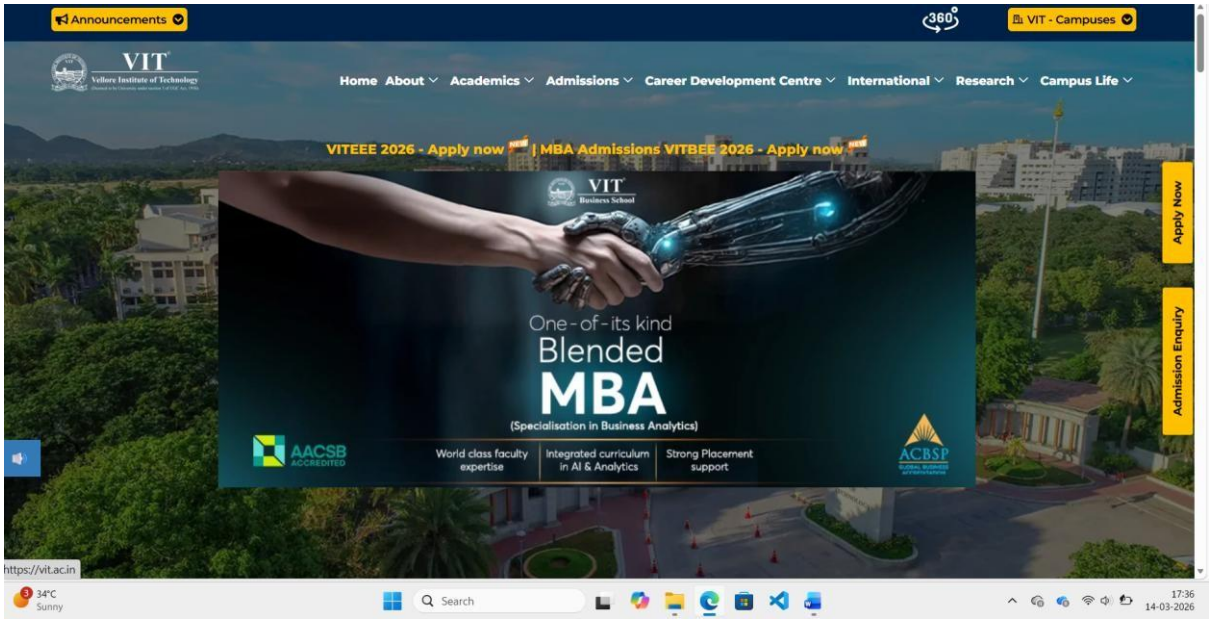
DOM-XSS: DOM stands for Document Object Model and is a programming interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document and this document can be either displayed in the browser window or as the HTML Source. Business Impact: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

Vulnerability path: https://vit.ac.in/

Vulnerability parameter: https://vit.ac.in/

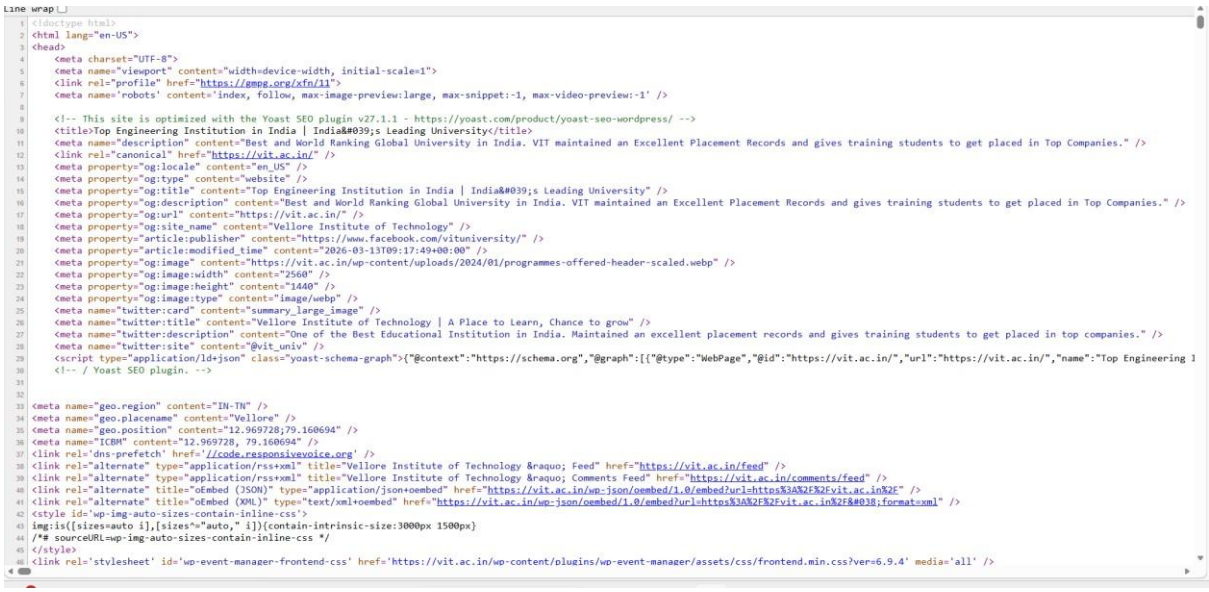
Steps to Reproduce:

- Access the URL



We will try to perform the DOM XSS attack which is displaying the source code of the webpage.

- Now we can access the source code of the webpage in two ways by clicking on the inspect option or by simply pressing Ctrl + U.



- Many programs can help to alter this source code to find vulnerable spots.

- So, getting access to the source code of the webpage can lead to potential vulnerabilities as attackers can abuse the loop holes, exploits, vulnerabilities in the code and act accordingly.

Recommendation:

- When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.

- Sanitize and validate all user inputs and ensure that any data displayed on the web page is properly encoded. Use output encoding libraries or functions to prevent script injection.

- Avoid using DOM manipulation methods that can introduce vulnerabilities. Be cautious with inner HTML, document. Write, and other methods that can execute scripts.

- Consider using well-established JavaScript frameworks and libraries that include built-in security features to prevent DOM XSS. 12.

Vulnerability name: JQuery 1.2 < 3.5.0 Multiple XSS CWE: 79 OWASP category: A03: Injections Description: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. Stored XSS (Persistent XSS): In this type of attack, the malicious script is permanently stored on a server or in a database, often in user-generated content like comments or forum posts. When other users view the affected content, the script executes in their browsers, potentially compromising their data or sessions.

Reflected XSS: Reflected XSS occurs when the malicious script is embedded in a URL and immediately executed when a victim clicks on a manipulated link. The script is not stored on a server but is reflected off a web application, making it a one-time attack.

DOM-based XSS: DOM stands for Document Object Model and is a programming interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document and this document can be either displayed in the browser window or as the HTML Source.

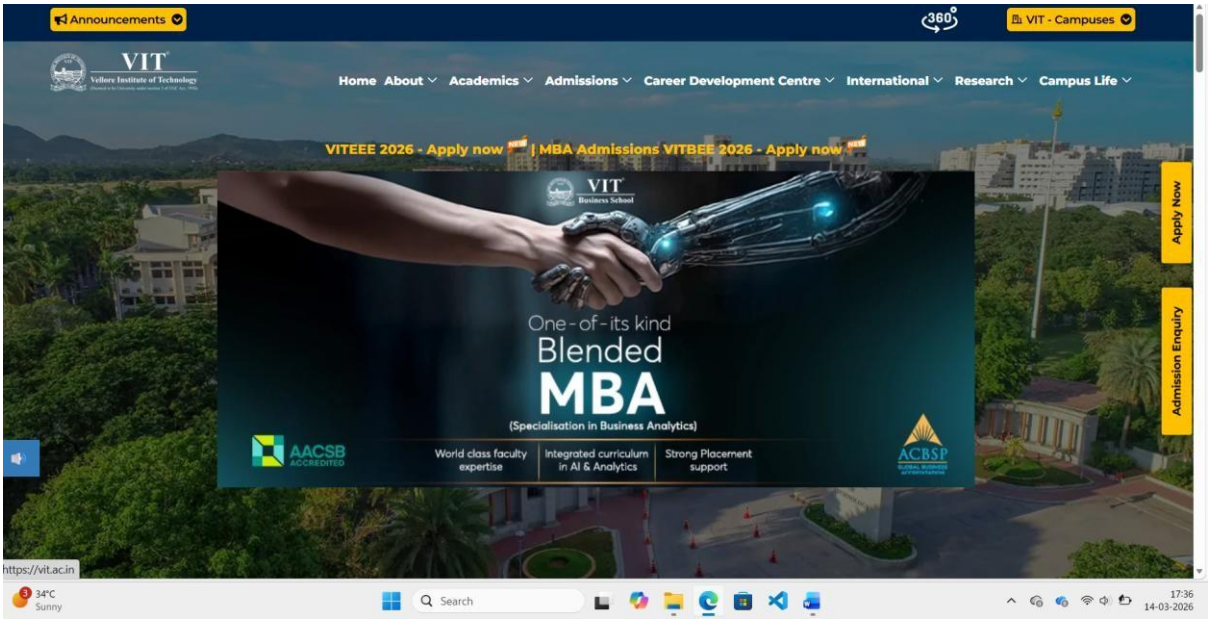
Business Impact: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker

may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

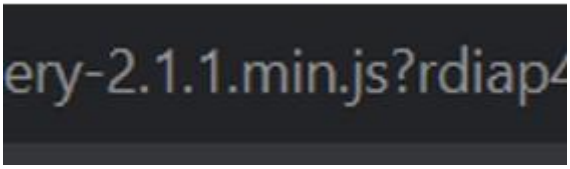
Vulnerability path :https://vit.ac.in/

Vulnerability parameter:
https://vit.ac.in/sites/all/themes/vittheme/js/jquery2.1.1.min.js?rdiap4 **Steps to Reproduce:**

- Access the URL



- Try to access the URL https://vit.ac.in/sites/all/themes/vittheme/js/jquery- .
  This will lead us to the webpage with jquery script in it.



- We can access the source code of the jquery script that has been used in making the website, which can lead to multiple potential threats

Recommendation:

- The most important recommendation is to update to the latest version of jQuery. Newer versions typically include security patches, bug fixes, and performance improvements.
- Stay vigilant about jQuery updates and security announcements. Subscribe to relevant security mailing lists or forums to receive notifications about new releases.

- A Content Security Policy (CSP) can help mitigate various types of client-side vulnerabilities, including Cross-Site Scripting (XSS). Consider implementing a CSP in your web application to restrict the sources from which scripts can be loaded.
- If updating jQuery is not immediately possible, review your code for potential security issues. Ensure that you are escaping or sanitizing user inputs and that you are using safe coding practices to prevent XSS and other security vulnerabilities.

13. Vulnerability name: Information Exposure (Web Server HTTP Header Internal IP Disclosure) CWE: 200

Description: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

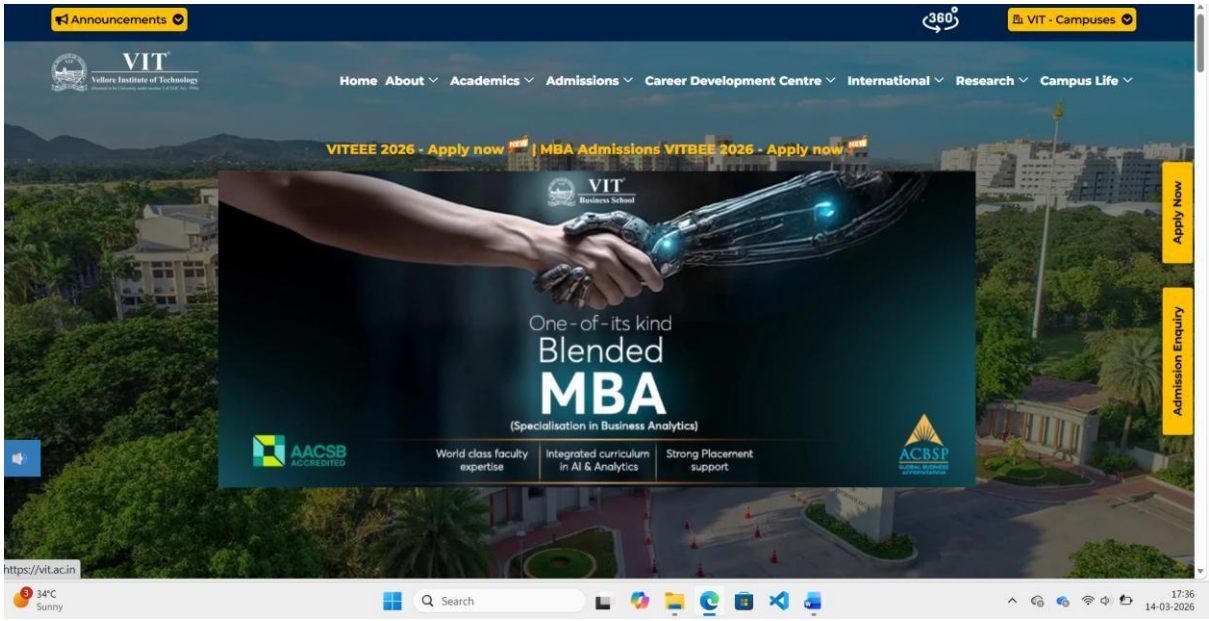
Business Impact: To effectively reduce the business consequences associated with CWE-200, organizations must prioritize the implementation of robust data access controls, secure data handling practices, and privacy protection measures. Regular security assessments and testing are instrumental in detecting and rectifying vulnerabilities related to information exposure. Furthermore, strict adherence to data protection regulations and industry standards is indispensable. These actions are of paramount importance in the protection of sensitive data, the preservation of user trust, and the upholding of the organization's esteemed reputation.

Vulnerability path: https://vit.ac.in/

Vulnerability parameter: https://vit.ac.in/

Steps to Reproduce:

- Access the URL



- We will be using a tool to perform a scan on this website. (Nessus Scanner)

LOW **Web Server HTTP Header Internal IP Disclosure**

Description

This may expose internal IP addresses that are usually hidden or masked behind a Network Address Translation (NAT) Firewall or proxy server.

There is a known issue with Microsoft IIS 4.0 doing this in its default configuration. This may also affect other web servers, web applications, web proxies, load balancers and through a variety of misconfigurations related to redirection.

Solution

```
Nessus was able to exploit the issue using the following request :

GET / HTTP/1.0
Accept-Charset: iso-8859-1,utf-8;q=0.9,*;q=0.1
Accept-Language: en
Connection: Close
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0)
Pragma: no-cache
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, image/png, */*



This produced the following truncated output (limited to 10 lines) :
------------------------------ snip ------------------------------
Location: https://10.10.7.35/
Content-Length: 0
```

- Now this image tells us the internal IP address of the target website, the attackers can easily exploit the website using the ip address.

Recommendation:

- Carefully review your web server's configuration files (e.g., Apache's httpd.conf, Nginx's nginx.conf) to identify anywhere internal IP addresses are inadvertently included in HTTP response headers.

- Remove or replace any internal IP addresses in the server configuration files with appropriate placeholders or public IP addresses.

- If you use proxy servers or load balancers, ensure that they are configured to hide internal IP addresses and only expose public IP addresses instances in HTTP headers.

- Review and configure HTTP response headers (e.g., Server, X-Powered-By) to limit the exposure of server and technology details, reducing the potential for disclosing internal IP addresses.

14. Vulnerability name: Information Exposure (SSL (Multiple Issues)) CWE: 200

Description: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

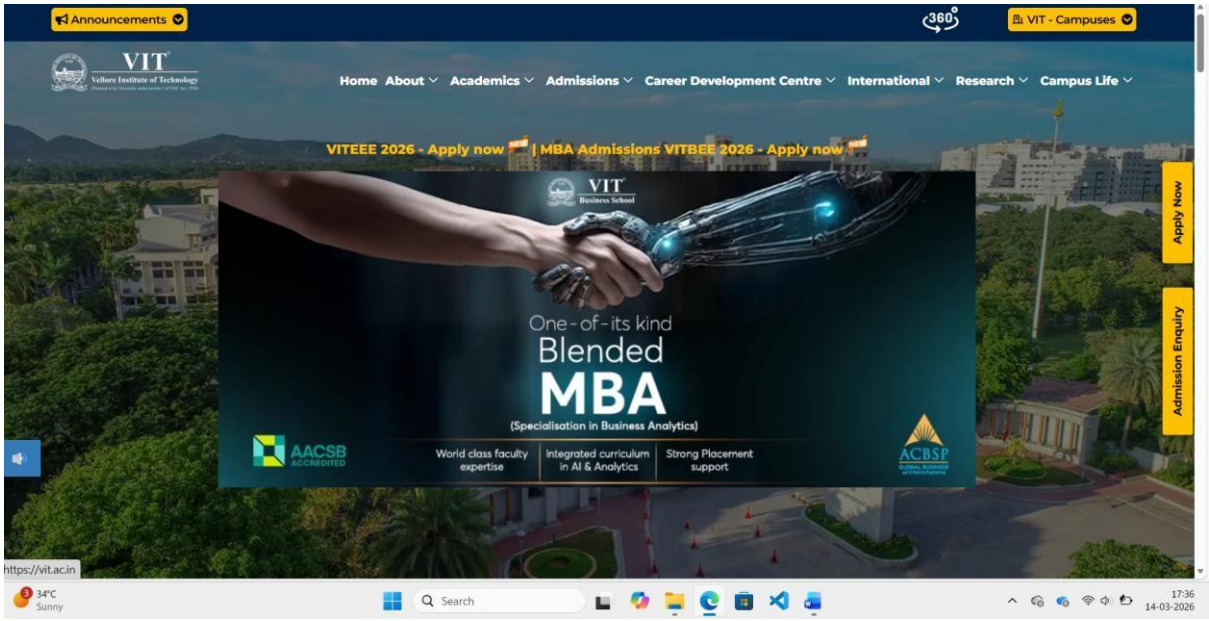
Business Impact: To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration, conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.

Vulnerability path: https://vit.ac.in/

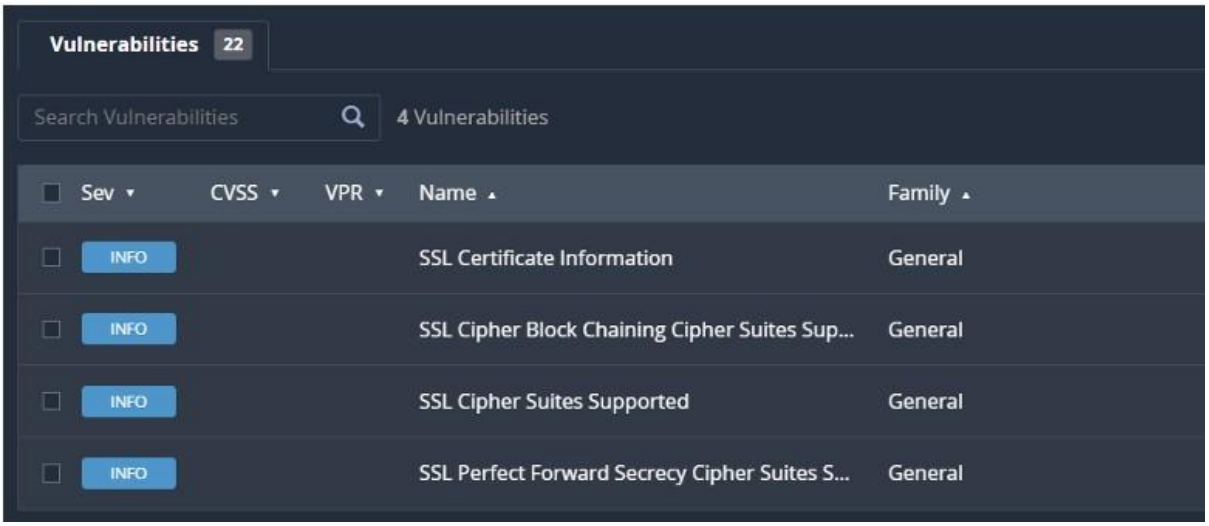
Vulnerability parameter: https://vit.ac.in/

Steps to Reproduce:

- Access the URL



- We will be using a tool to perform a scan on this website. (Nessus Scanner)
- Through this tool we can see the vulnerability in the scan along with detailed information regarding that vulnerability.
- Now this image tells us the type of encryption used in the target website with the public key details as well.
- Through this we can understand what different types of encryptions are used in the website.



INFO

SSL Certificate Information

Description

This plugin connects to every SSL-related port and attempts to extract and dump the X.509 certificate.

Output

```
Subject Name:

Common Name: *.vit.ac.in

Issuer Name:

Country: GB
State/Province: Greater Manchester
Locality: Salford
Organization: Sectigo Limited
Common Name: Sectigo RSA Domain Validation Secure Server CA

Serial Number: 46 53 67 B6 23 C5 BE EE B9 6E E2 C0 5F 46 C9 F7

Version: 3

Signature Algorithm: SHA-256 With RSA Encryption

Not Valid Before: Sep 04 00:00:00 2023 GMT
Not Valid After: Aug 03 23:59:59 2024 GMT

Public Key Info:

Algorithm: RSA Encryption
```

INFO

SSL Cipher Block Chaining Cipher Suites Supported

Description

The remote host supports the use of SSL ciphers that operate in Cipher Block Chaining (CBC) mode. These cipher suites offer additional security over Electronic Codebook (ECB) mode, but have the potential to leak information if used improperly.

INFO

SSL Cipher Suites Supported

Description

This plugin detects which SSL ciphers are supported by the remote service for encrypting communications.

INFO

SSL Perfect Forward Secrecy Cipher Suites Supported

Description

The remote host supports the use of SSL ciphers that offer Perfect Forward Secrecy (PFS) encryption. These cipher suites ensure that recorded SSL traffic cannot be broken at a future date if the server's private key is compromised.

```
Here is the list of SSL CBC ciphers supported by the remote server :

  High Strength Ciphers (>= 112-bit key)

    Name                        Code            KEX          Auth     Encryption             MAC
    ---------------------       ----------      ---          ----     --------------------   ---
    ECDHE-RSA-CAMELLIA-CBC-128  0xC0, 0x76      ECDH         RSA      Camellia-CBC(128)      SHA256
    ECDHE-RSA-CAMELLIA-CBC-256  0xC0, 0x77      ECDH         RSA      Camellia-CBC(256)      SHA384
    DHE-RSA-AES128-SHA          0x00, 0x33      DH           RSA      AES-CBC(128)           SHA1
    DHE-RSA-AES256-SHA          0x00, 0x39      DH           RSA      AES-CBC(256)           SHA1
    DHE-RSA-CAMELLIA128-SHA     0x00, 0x45      DH           RSA      Camellia-CBC(128)      SHA1
    DHE-RSA-CAMELLIA256-SHA     0x00, 0x88      DH           RSA      Camellia-CBC(256)      SHA1
    DHE-RSA-SEED-SHA            0x00, 0x9A      DH           RSA      SEED-CBC(128)          SHA1
    ECDHE-RSA-AES128-SHA        0xC0, 0x13      ECDH         RSA      AES-CBC(128)           SHA1
    ECDHE-RSA-AES256-SHA        0xC0, 0x14      ECDH         RSA      AES-CBC(256)           SHA1
    AES128-SHA                  0x00, 0x2F      RSA          RSA      AES-CBC(128)           SHA1
    AES256-SHA                  0x00, 0x35      RSA          RSA      AES-CBC(256)           SHA1
    CAMELLIA128-SHA             0x00, 0x41      RSA          RSA      Camellia-CBC(128)      SHA1
    CAMELLIA256-SHA             0x00, 0x84      RSA          RSA      Camellia-CBC(256)      SHA1
    SEED-SHA                    0x00, 0x96      RSA          RSA      SEED-CBC(128)          SHA1
    DHE-RSA-AES128-SHA256       0x00, 0x67      DH           RSA      AES-CBC(128)           SHA256
    DHE-RSA-AES256-SHA256       0x00, 0x6B      DH           RSA      AES-CBC(256)           SHA256
    DHE-RSA-CAMELLIA128-SHA256  0x00, 0xBE      DH           RSA      Camellia-CBC(128)      SHA256
    DHE-RSA-CAMELLIA256-SHA256  0x00, 0xC4      DH           RSA      Camellia-CBC(256)      SHA256
    ECDHE-RSA-AES128-SHA256     0xC0, 0x27      ECDH         RSA      AES-CBC(128)           SHA256
    ECDHE-RSA-AES256-SHA384     0xC0, 0x28      ECDH         RSA      AES-CBC(256)           SHA384
    RSA-AES128-SHA256           0x00, 0x3C      RSA          RSA      AES-CBC(128)           SHA256
    RSA-AES256-SHA256           0x00, 0x3D      RSA          RSA      AES-CBC(256)           SHA256
    RSA-CAMELLIA128-SHA256      0x00, 0xBA      RSA          RSA      Camellia-CBC(128)      SHA256
    RSA-CAMELLIA256-SHA256      0x00, 0xC0      RSA          RSA      Camellia-CBC(256)      SHA256
```

Recommendation:

- Configure your web server to use the latest and strongest encryption protocols, such as TLS 1.2 or higher. Disable outdated and insecure protocols like SSLv3.
- Implement Perfect Forward Secrecy to ensure that even if the private key is compromised, previously recorded encrypted traffic remains secure.
- Safeguard your private keys by storing them securely and ensuring they are not accessible to unauthorized parties.

15. Vulnerability name: Information Exposure (HTTP (Multiple Issues)) CWE: 200 Description: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

Business Impact: To effectively reduce the business repercussions of HTTP issues, organizations must place a strong emphasis on ongoing monitoring and enhancement of their web services. Consistent performance testing and security assessments, coupled with proactive maintenance, are indispensable to ensure a seamless and secure online presence. Furthermore, offering timely and helpful customer support and transparent communication when HTTP issues arise is instrumental in mitigating adverse effects on user trust and satisfaction.

Vulnerability path: https://vit.ac.in/

Vulnerability parameter: https://vit.ac.in/

Steps to Reproduce:

- Access the URL
- We will be using a tool to perform a scan on this website. (Nessus Scanner)
- From the above image we can see the details of protocol version of the website and its status.
- We can see the type of remote webserver here.
- All this information can lead to multiple potential threats such as data breaches by hackers.

Recommendation:

- Minimize the use of unnecessary redirects as they can increase page load times. Use 301 (permanent) redirects for consistent content.
- Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.
- Properly configure CORS to control which domains are allowed to make requests to your server.
- Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.

16. Vulnerability name: OS Identification Failed CWE: 200 Description: OS identification failed refers to the inability of a system or software to accurately recognize the operating system running on a device or network. This issue can result from various factors such as misconfigurations, network issues, or compatibility problems. The failure to identify the OS correctly can lead to security vulnerabilities, hindered software updates, and potential operational disruptions. Resolving this issue promptly is crucial to ensure accurate security measures and the smooth functioning of the system or network.

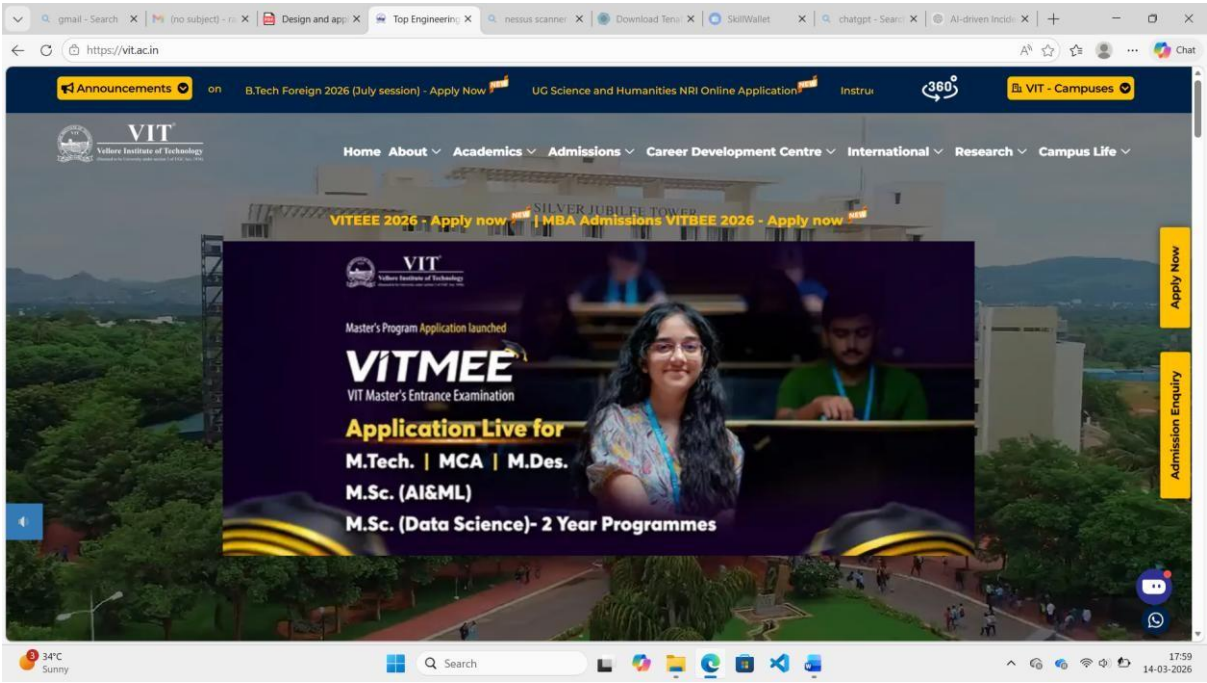
Business Impact: It may lead to operational inefficiencies, potential security vulnerabilities, non-compliance with industry regulations, and a possible loss of customer trust. Such failures could result in downtime, compromised data security, regulatory penalties, and damage to the company's reputation, potentially leading to financial losses and decreased market competitiveness. Resolving this issue promptly is crucial to mitigate these risks and maintain the smooth functioning of the organization.

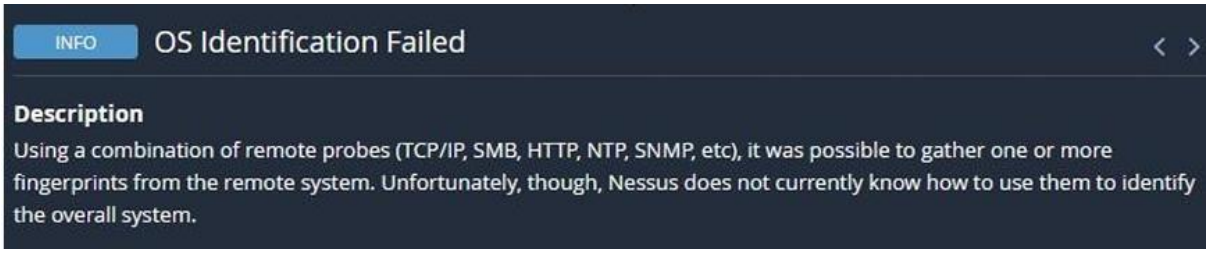
Vulnerability path: https://vit.ac.in/

Vulnerability parameter: https://vit.ac.in/

Steps to Reproduce:

➢ Access the URL





➢ From the above image, we can see the details of SSL Certificate Information and SinFP Information.

➢ If it is released without proper authorization, it can lead to serious consequences such as increased vulnerability to cyberattacks, potential data breaches, privacy violations, and legal repercussions.

➢ All this information can lead to multiple potential threats such as data breaches by hackers.

Recommendations:

- Employ robust data encryption methods for both data in transit and at rest.
- Implement strict access controls and authentication protocols to limit unauthorized access.
- Use secure communication channels, such as encrypted email and file transfer protocols.
- Provide comprehensive training to employees on data security best practices.
- Conduct regular security audits and assessments to identify and address vulnerabilities promptly.
- Ensure compliance with relevant data protection regulations and have a well-defined incident response plan in place.

NESSUS SCAN REPORT

Vulnerabilities Total: 28

SEVERITY	CVSS V3.0	VPR SCORE	PLUGIN	NAME
MEDIUM	6.5	-	142960	HSTS Missing From HTTPS Server (RFC 6797)
MEDIUM	6.5	-	104743	TLS Version 1.0 Protocol Detection
MEDIUM	6.5	-	157288	TLS Version 1.1 Protocol Deprecated
LOW	3.7	4.5	83875	SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)
INFO	N/A	-	46180	Additional DNS Hostnames
INFO	N/A	-	39446	Apache Tomcat Detection
INFO	N/A	-	45590	Common Platform Enumeration (CPE)
INFO	N/A	-	54615	Device Type
INFO	N/A	-	84502	HSTS Missing From HTTPS Server
INFO	N/A	-	10107	HTTP Server Type and Version
INFO	N/A	-	24260	HyperText Transfer Protocol (HTTP) Information
INFO	N/A	-	11219	Nessus SYN scanner
INFO	N/A	-	19506	Nessus Scan Information
INFO	N/A	-	11936	OS Identification
INFO	N/A	-	10919	Open Port Re-check
INFO	N/A	-	56984	SSL / TLS Versions Supported
INFO	N/A	-	10863	SSL Certificate Information
INFO	N/A	-	95631	SSL Certificate Signed Using Weak Hashing Algorithm (Known CA)
INFO	N/A	-	70544	SSL Cipher Block Chaining Cipher Suites Supported

INFO	N/A	-	21643	SSL Cipher Suites Supported
INFO	N/A	-	57041	SSL Perfect Forward Secrecy Cipher Suites Supported
INFO	N/A	-	94761	SSL Root Certification Authority Certificate Information
INFO	N/A	-	156899	SSL/TLS Recommended Cipher Suites

vit.ac.in

0	0	1	1	26
CRITICAL	HIGH	MEDIUM	LOW	INFO

Vulnerabilities Total: 28

SEVERITY	CVSS V3.0	VPR SCORE	PLUGIN	NAME
MEDIUM	6.1	5.7	136929	JQuery 1.2 < 3.5.0 Multiple XSS
LOW	3.1	2.2	10759	Web Server HTTP Header Internal IP Disclosure
INFO	N/A	-	48204	Apache HTTP Server Version
INFO	N/A	-	45590	Common Platform Enumeration (CPE)
INFO	N/A	-	10107	HTTP Server Type and Version
INFO	N/A	-	24260	HyperText Transfer Protocol (HTTP) Information
INFO	N/A	-	106658	JQuery Detection
INFO	N/A	-	11219	Nessus SYN scanner
INFO	N/A	-	19506	Nessus Scan Information
INFO	N/A	-	42823	Non-compliant Strict Transport Security (STS)
INFO	N/A	-	50350	OS Identification Failed
INFO	N/A	-	66334	Patch Report
INFO	N/A	-	56984	SSL / TLS Versions Supported
INFO	N/A	-	10863	SSL Certificate Information
INFO	N/A	-	70544	SSL Cipher Block Chaining Cipher Suites Supported
INFO	N/A	-	21643	SSL Cipher Suites Supported
INFO	N/A	-	57041	SSL Perfect Forward Secrecy Cipher Suites Supported
INFO	N/A	-	94761	SSL Root Certification Authority Certificate Information
INFO	N/A	-	156899	SSL/TLS Recommended Cipher Suites

Stage 2: Nessus Vulnerability Assessment

With the Nessus scan report now available, the project team possesses critical insights into existing vulnerabilities and potential threats within the target environment. This information forms the cornerstone for the planning and design of our AI-driven incident response platform, enabling a strategy that is informed, methodical, and aligned with organizational security priorities.

2.1 Overview of Nessus Scanning

Nessus is a widely utilized vulnerability scanning tool that enables organizations to systematically identify security weaknesses across networks, systems, and applications. The Nessus scan process typically involves the following key stages:

2.1.1 Preparation

- **Define Objectives:** Establish the scope of the scan by specifying which systems, networks, or applications will be assessed and the types of vulnerabilities to be detected.
- **System Access:** Ensure necessary permissions and credentials are available to scan the targeted infrastructure.

2.1.2 Configuration

- **Scan Policy Setup:** Configure a scan policy detailing the types of vulnerabilities to evaluate, scanning schedules, and specific plugin configurations.
- **Target Assets Definition:** Identify IP addresses, hostnames, or network ranges to be included in the scan.

2.1.3 Scanning Execution

- **Initiate Scan:** Launch the scan using the predefined policy. Nessus systematically examines the target systems using its extensive plugin library.
- **Vulnerability Detection:** Nessus identifies vulnerabilities, misconfigurations, open ports, and other security issues. Vulnerabilities are categorized by severity to prioritize remediation efforts.

2.1.4 Reporting

- **Report Generation:** Nessus produces detailed reports summarizing the discovered vulnerabilities, their severity, remediation recommendations, and supporting evidence. Reports can be exported in formats such as PDF, HTML, or CSV.

2.1.5 Remediation

- **Review Findings:** IT and security teams analyze the scan report to determine remediation priorities.
- **Remediation Steps:** Apply necessary corrective actions, such as patching software, updating configurations, or enforcing security policies.

2.1.6 Follow-Up

- **Regular Scanning:** Conduct periodic Nessus scans to continuously monitor system security posture.
- **Integration:** Integrate Nessus into broader security workflows to automate vulnerability management and streamline response to emerging threats.
- **Compliance Auditing:** Verify adherence to applicable security standards and regulations.
- **Documentation & Training:** Maintain thorough records of scans, remediation actions, and compliance checks. Ensure teams are trained in Nessus usage and report interpretation.

2.2 Key Features of Nessus

1. **Vulnerability Assessment:** Identifies missing patches, weak passwords, misconfigurations, and other exploitable weaknesses.
2. **Network Scanning:** Evaluates servers, routers, switches, firewalls, and network services.
3. **Plugin Architecture:** Leverages regularly updated plugins to detect newly discovered vulnerabilities.

4. **Compliance Auditing:** Supports regulatory and security standards such as PCI DSS, CIS benchmarks.
5. **Customization:** Allows scan policies to be tailored according to organizational requirements.
6. **Reporting:** Provides detailed vulnerability reports with severity ratings and remediation guidance.
7. **Integration:** Can be integrated with other security tools for automated vulnerability management.
8. **User-Friendly Interface:** Web-based interface accessible to both security experts and IT professionals.
9. **Licensing Options:** Offers both free and commercial versions with varying functionality.

Note: Nessus identifies vulnerabilities but does not remediate them; corrective actions must be implemented by IT and security teams.

2.3 Target and Scan Details

- **Target Website:** http://testfire.net
- **Target IP Address:** 54.83.41.36
- **Scan Type:** Web Application and Network Vulnerability Assessment

---

2.4 Sample Vulnerability Findings

S. No	Vulnerability Name	Severity	Plugins / Notes
1	Cross-Site Scripting (Stored)	Low	Web Application Scanner
2	Broken Access Control	Medium	SSH Weak HMAC Algorithms Supported

3	SQL Injection	Medium	N/A
4	Broken Authentication	Low	HSTS Missing from Server
5	Insecure Direct Object Reference	Info	Service Detection
6	Security Misconfiguration	Low	SSL Cipher Block Chaining Suites
7	Cross-Site Request Forgery (CSRF)	Low	HTTP Form-Based Authentication Detected
8	Cleartext Transmission of Sensitive Info	Medium	Transmission over HTTP
9	ClickJacking	Info	HTML Rendering Layer Vulnerability
10	Webserver Allows Password Auto-Completion	Low	SSL/TLS Diffie-Hellman Modulus ≤1024 bits (Logjam)

2.4.1 Vulnerability Example: Cross-Site Scripting (Stored)

- **Severity:** Low
- **Plugin:** Web Application Scanner
- **Description:** Occurs when malicious scripts are injected into web applications, potentially executing in the browsers of other users. Reflected XSS reflects input in responses, whereas stored XSS persists in databases or logs.
- **Solution:**

  ○ Use parameterized queries or prepared statements to prevent injection. ○ Sanitize and validate all user input. Employ output encoding libraries to neutralize scripts.
- **Business Impact:** Malicious scripts can target high-privilege users or sensitive data, potentially enabling attackers to execute privileged operations or gain unauthorized access.

2.4.2 Vulnerability Example: Broken Access Control

- **Severity:** Medium
- **Plugin:** SSH Weak HMAC Algorithms Supported
- **Description:** Inconsistent or missing access control checks allow unauthorized access to sensitive data or operations. Consequences include data exposure, arbitrary code execution, and operational disruption.
- **Solution:**
  - Enforce robust authentication and authorization mechanisms.
  - Implement secure session management with strong session IDs, timeouts, and token regeneration.
- **Business Impact:** Can lead to financial losses, reputational damage, regulatory violations, and operational disruption.

2.5 Detailed Vulnerability Findings

2.5.1 SQL Injection (CWE-89)

- **Severity:** Medium
- **Plugin:** N/A (typically detected via security testing tools)
- **Port:** 80 / 443
- **Description:** The application constructs SQL queries using user-supplied input without proper sanitization or neutralization of special characters. This allows attackers to manipulate the SQL commands and potentially access or modify sensitive data.
- **Solution:**
  - Use **parameterized statements** or **prepared statements** to prevent injection.
  - Employ **stored procedures** to separate SQL logic from application code.
  - Apply strict **input validation** to ensure user input conforms to expected formats.

- **Business Impact:** SQL injection can lead to data breaches, financial losses, reputational damage, legal implications, and operational disruptions. Remediation is critical for maintaining application and data security.

2.5.2 Broken Authentication (CWE-285)

- **Severity:** Low
- **Plugin:** HSTS Missing from HTTPS Server (plugin id: 142960)
- **Port:** 443
- **Description:** Weak authentication allows users to gain unauthorized access to other users' data by manipulating session keys or credentials.
- **Solution:**
    - Enforce **strong authentication mechanisms** such as MFA or 2FA.
    - Implement **role-based access control (RBAC)** to ensure users have appropriate permissions.
    - Deploy **IDS/IPS** systems to detect and prevent unauthorized access attempts.
- **Business Impact:** Weak authentication exposes sensitive data and user identities, potentially leading to reputational damage, regulatory penalties, and operational disruption.

2.5.3 Insecure Direct Object Reference (IDOR, CWE-639)

- **Severity:** Info
- **Plugin:** Service Detection (plugin id: 22964)
- **Port:** 80 / 443
- **Description:** IDOR arises when attackers can access or modify objects by manipulating identifiers (e.g., URL parameters), bypassing access controls.
- **Solution:**

- Implement **parameterized queries** or prepared statements. ○ Enforce **access control checks** for all sensitive resources. ○ Introduce account lockout policies to mitigate brute force attempts.

- **Business Impact:** Exploitation can compromise sensitive data, threaten system integrity, and damage organizational reputation.

2.5.4 Security Misconfiguration (CWE-732)

- **Severity:** Low
- **Plugin:** SSL Cipher Block Chaining Cipher Suites
- **Port:** 443
- **Description:** Security-critical resources are accessible due to improper configuration, allowing unauthorized modifications.
- **Solution:**
  - Document all system changes using a **change management process**.
  - Follow vendor-recommended **secure configuration guidelines**. ○ Conduct **regular compliance checks** against industry standards.
- **Business Impact:** Misconfigurations can expose sensitive data and weaken overall system security.

2.5.5 Cross-Site Request Forgery (CSRF, CWE-352)

- **Severity:** Low
- **Plugin:** HTTP Form-Based Authentication Detected
- **Port:** 8080 / 443

- **Description:** CSRF forces users to execute unintended actions while authenticated. Attackers may manipulate users into performing state-changing actions like transferring funds or modifying account settings.
- **Solution:**
    - Implement **Synchronizer Token Patterns** with unique, per-request tokens. ○ Use the **SameSite attribute** in cookies.
    - Leverage framework-specific CSRF protections.
    - Apply a **Content Security Policy** to restrict untrusted content.
- **Business Impact:** CSRF can lead to unauthorized transactions, data tampering, and compromise of administrative accounts, risking financial and reputational harm.

2.5.6 Cleartext Transmission of Sensitive Information (CWE-319)

- **Severity:** Medium
- **Plugin:** Cleartext Transmission over HTTP
- **Port:** 443
- **Description:** Sensitive data is transmitted without encryption, allowing adversaries to intercept information during transit.
- **Solution:**
    - Encrypt data in transit using **TLS/SSL**. ○ Educate users on secure practices (recognizing HTTPS, avoiding public Wi-Fi). ○ Implement **data masking** or redaction during transmission.
- **Business Impact:** Unencrypted transmissions can lead to data leakage, regulatory violations, and loss of customer trust.

2.5.7 Clickjacking (CWE-1021)

- **Severity:** Info
- **Plugin:** HTML Rendering Layer Vulnerability
- **Port:** 80 / 443
- **Description:** The application does not prevent embedding of its pages in frames, allowing attackers to trick users into interacting with malicious interfaces.
- **Solution:**
    - Set **X-Frame-Options** headers to DENY or SAMEORIGIN.
    - Implement **frame-busting scripts**.
    - Apply a **Content Security Policy** for iframe restrictions.
- **Business Impact:** Clickjacking can deceive users, compromise credentials, and erode user trust.

2.5.8 Web Server Allows Password Auto-Completion (CWE-310)

- **Severity:** Low
- **Plugin:** SSL/TLS Diffie-Hellman Modulus ≤ 1024 Bits (Logjam)
- **Port:** 443
- **Description:** Weak password storage or auto-completion mechanisms compromise data confidentiality.
- **Solution:**
    - Use **strong salted password hashing algorithms**. ○ Implement **robust encryption key management**.
    - Employ secure **TLS/SSL protocols** for data transmission.
- **Business Impact:** Weak cryptography can expose credentials, leading to breaches and reputational loss.

2.5.9 Additional Vulnerabilities

S. No	Vulnerability Name	Severity	Plugins / Notes
1	Cross-site scripting (DOM-based XSS)	Medium	Multiple XSS
2	jQuery 1.3 < 3.5.0 Multiple XSS	Medium	CGI Abuse: XSS
3	Information Exposure (Web Server HTTP Header Internal IP)	Low	Webserver HTTP Header Internal IP Disclosure
4	Information Exposure (SSL – Multiple Issues)	Info	SSL Certificate Information
5	Information Exposure (HTTP – Multiple Issues)	Info	HTTP Information
6	OS Identification Failed	Info	TCP/IP Remote Probing

Stage 3: Implementation of Zscaler Private Access & Future Scope

3.1 Overview

The implementation of **Zscaler Private Access (ZPA)** represents a strategic move from a traditional, network-centric security model to a **resilient, identity-centric Zero Trust architecture**. By enforcing **least-privilege access** and rendering private applications invisible to the internet, this solution neutralizes lateral movement and network-based threats.

Key benefits include:

- **Enhanced security posture** against modern cyber threats.
- **Seamless, high-performance experience** for distributed workforces.
- **Simplified IT operations**, reducing reliance on legacy VPN infrastructure.
- **Granular visibility** into user-application interactions.
- Establishes a **future-ready foundation** for secure digital transformation.

3.2 Future Scope

3.2.1 Stage 1 – Future Scope of Web Application Testing

The demand for web application testing continues to grow as internet adoption and web-based services expand. Key aspects include:

- **Increased Demand:** Growth of online services increases the need for secure, reliable web applications.
- **Advanced Technologies:** Integration of AI, IoT, and blockchain increases testing complexity.
- **Security Emphasis:** Rising cyber threats make security testing crucial for compliance (e.g., GDPR).
- **Mobile and Cross-Browser Testing:** Ensuring responsiveness across devices and browsers.
- **Performance Testing:** Load, stress, and performance testing to meet user expectations.

3.2.2 Stage 2 – Future Scope of Vulnerability Testing

Vulnerability identification in web applications is rapidly evolving. Key emerging trends:

- **AI-Driven Testing:** AI and ML tools will scan code and data to identify vulnerabilities faster than manual testing.
- **IoT and Mobile App Testing:** Expanding to cover connected devices and mobile platforms.
- **Blockchain & Cryptocurrency Security:** Specialized testing for blockchain-based applications and digital assets.
- **Serverless Architecture Testing:** Assessing security of serverless functions and APIs.
- **Container Security:** Securing Docker/Kubernetes containers and orchestration configurations.

3.2.3 Stage 3 – Future Scope of Aware Access Control (ZPA)

ZPA deployment enables a mature security ecosystem. Future enhancements include:

- **Integration with Zscaler Internet Access (ZIA):** Unified **SASE platform** to manage web and private traffic with a single policy framework.
- **Advanced Device Posture Checking:** Real-time risk assessment using EDR tools to block compromised devices.
- **Privileged Remote Access:** Time-bound, auditable access for vendors and contractors, replacing jump boxes.
- **IoT/OT Segmentation:** Micro-tunneling to restrict IoT and OT device communications.
- **AI-Driven Anomaly Detection:** Machine learning on ZPA logs to detect unusual user behavior and trigger automated MFA or access revocation.
- **Cloud Workload Protection:** Secure container and microservice connections across multi-cloud environments (AWS, Azure, GCP).
- **Automated Provisioning/Deprovisioning:** Immediate access revocation upon employee role changes or termination.
- **Digital Experience Monitoring (DEM):** Proactively monitor end-user experience, identifying latency or performance issues before impacting productivity.

3.3 Topics Explored

1. Zscaler
2. Zero Trust Security
3. Zscaler Private Access (ZPA)
4. Virtual Private Networks (VPNs)
5. Malware Information Sharing Platform (MISP)
6. College Network Information
7. Security Operations Center (SOC) Deployment in the College
8. Threat Intelligence
9. Incident Response

3.4 Tools Explored

- **Nessus Vulnerability Scanner** – For identifying vulnerabilities in the college network and web applications.